0xchou00 Detection Tool: Architecture, Detection Flow, Enrichment, and Correlation

Technical documentation for the public codebase

April 17, 2026

Abstract

0xchou00 is intentionally not presented as a full SIEM. It is a lightweight detection tool designed for single-node operation, direct inspection, and low-friction deployment. This document explains the codebase as it exists publicly: local and API-based ingestion, normalization, IP enrichment, event-level detection, alert correlation, SQLite persistence, tamper-evident integrity, and an operator dashboard. It also explains the design boundaries that distinguish a detection tool from a large-scale SIEM deployment.

1 Detection Tool vs. SIEM

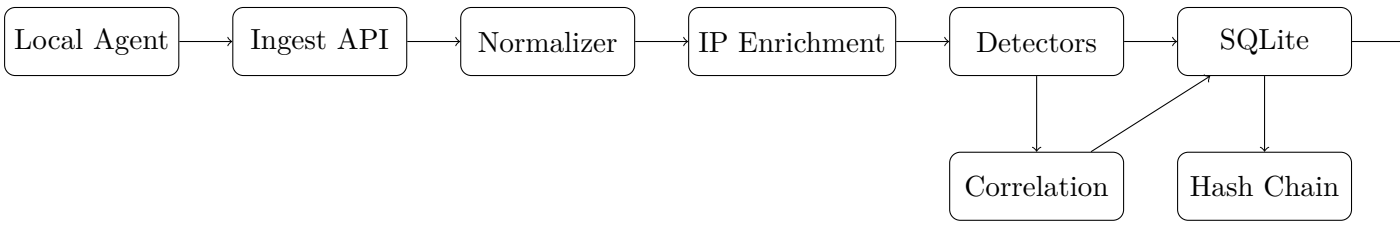
A SIEM usually addresses large-scale aggregation, indexing, retention, search, and cross-source analytics across many systems. NIST log-management guidance emphasizes broad collection, storage, review, and analysis processes across an organization's infrastructure [1]. Products such as Splunk, Elastic-based stacks, and Wazuh typically add distributed storage, scaling controls, richer search, and fleet-wide collection workflows.

0xchou00 solves a narrower problem:

- collect security-relevant logs on one node;
- normalize them into one event model;
- enrich source IPs with local or cached context;
- apply explainable detector logic;
- correlate short multi-step attack patterns;
- preserve the result in a tamper-evident local store.

That is why the repository is described as a **security detection tool**. The choice is technical, not cosmetic.

2 Architecture



This pipeline is linear on purpose. Each stage has a narrow responsibility, which keeps the code inspectable and keeps failures localized.

3 Log Processing Pipeline

3.1 Ingestion

The tool accepts log lines in two ways:

- `POST /ingest` for direct API submission;
- a local Python agent that tails `/var/log/auth.log` and `/var/log/nginx/access.log`.

The agent uses offset and inode tracking so it can resume after restarts and recover from common log-rotation behavior. This is closer to a local collector than a demonstration script.

3.2 Normalization

Supported public parsers:

- SSH authentication failures and successes from `sshd`;
- Apache or Nginx combined-style web access logs.

The result is a single `LogEvent` schema with stable fields such as source type, event type, source IP, HTTP metadata, and severity. This keeps the detector layer source-aware only where necessary.

4 Enrichment Strategy

4.1 Local-first Context

The enrichment layer runs immediately after normalization. It can add:

- `country`;
- `is_malicious`;
- `reputation_score`;
- `enrichment_source`.

The order is intentional:

1. static blacklist;
2. local GeoIP database;
3. cached threat-intelligence data;
4. asynchronous AbuseIPDB refresh when configured.

4.2 Why the External Lookup Is Asynchronous

The detection path should not pause because a remote reputation API is slow. Therefore, first-seen IPs are enriched with whatever local context is available immediately, and optional external reputation data is fetched in the background and cached for later events. This trades first-event completeness for stable ingestion latency.

5 Detection Logic

5.1 SSH Brute Force

The brute-force detector counts failed SSH authentications per source IP inside a 60-second sliding window:

$$\texttt{failures}_{ip,60s} \geq 5 \tag{1}$$

If the source IP is already flagged as malicious or reaches three times the threshold, the alert is raised to `critical`. This reflects the role of repeated authentication failures in brute-force activity [2].

5.2 Suspicious Web IP Behavior

The web detector covers three high-signal cases:

- excessive request rate;
- excessive error ratio;
- access to sensitive paths such as `/.env` or `/.git/config`.

These checks are aligned with common reconnaissance and forced-browsing patterns described by OWASP and ATT&CK scanning guidance [5, 3].

5.3 YAML Rules

YAML rules allow field equality, substring matches, regular expressions, and grouped thresholds without editing Python. This keeps detection development simple for narrow environments.

5.4 Frequency Anomaly

The anomaly detector builds a short baseline from recent fixed windows and alerts when the current event count exceeds:

$$\max(12,\ 3 \times \mu) \tag{2}$$

where μ is the average of the previous baseline windows. This is simple threshold-based anomaly detection, not a machine-learning subsystem.

6 Correlation Logic

Correlation is applied after detector alerts exist. It does not work on raw events; it works on recent alerts. Rules in `rules/correlation_rules.yml` specify:

- a set of required detector alerts;
- a time window;
- whether the source IP must match;
- output title, description, and severity.

The public examples include:

- SSH brute force combined with suspicious web probing from the same source;

- traffic anomaly combined with sensitive path access.

Correlated alerts are stored in the same alert table as detector alerts. This keeps the API and dashboard simple because there is no separate retrieval path.

7 Integrity

Each stored event and alert is sealed into a chain entry:

$$H_n = \mathrm{SHA256}(type_n \parallel id_n \parallel payload_n \parallel H_{n-1} \parallel t_n) \tag{3}$$

The hash primitive follows the Secure Hash Standard [4]. The higher-level design is consistent with the secure-audit-log pattern described by Schneier and Kelsey [6]. The tool verifies both chain continuity and current payload hashes against stored entities. This gives tamper evidence, not distributed consensus.

8 Security Practices

- API access uses role-ranked keys.
- Detector and correlation logic are data-driven where practical.
- Optional external threat intelligence is cached and bounded by timeout.
- The local agent tracks offsets to avoid replaying entire files after restart.
- Integrity verification is exposed through a dedicated endpoint.

9 Alternatives and Tradeoffs

9.1 ELK Stack

Elastic-based logging stacks provide scalable indexing, distributed search, and large-volume querying. They are stronger for broad telemetry retention and search, but heavier to operate than a single SQLite-backed node.

9.2 Wazuh

Wazuh brings host agents, rule content, and central security analytics. It is broader in host coverage, but it also introduces a larger deployment and management surface.

9.3 Splunk

Splunk is mature for enterprise log management and search, but it is designed for environments that need higher ingestion scale, search depth, and administrative control than this tool targets.

9.4 Why FastAPI and SQLite Here

FastAPI keeps the HTTP layer compact and explicit. SQLite keeps the storage path inspectable and easy to move. Those choices fit the actual scope of the codebase:

- one host;
- one operator or small lab;

- one local database;
- direct code inspection over opaque infrastructure.

10 Limitations

- only SSH and web access logs are normalized publicly;
- the dashboard is polling-based;
- SQLite is not intended for multi-node ingestion at scale;
- first-seen IPs may only have local context until the asynchronous cache refresh completes;
- the integrity chain cannot prevent destructive deletion by a privileged local user.

11 Conclusion

0xchou00 is most useful when treated honestly: a lightweight detection tool with real ingestion, enrichment, detection, correlation, and integrity features on a single node. Its main strength is that the entire path from raw line to correlated alert remains readable, debuggable, and easy to operate without external infrastructure.

References

[1] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Technical Report NIST SP 800-92, National Institute of Standards and Technology, 2006. URL https://csrc.nist.gov/pubs/sp/800/92/final.

[2] MITRE ATT&CK. Brute force, technique t1110, 2025. URL https://attack.mitre.org/techniques/T1110. Accessed 2026-04-17.

[3] MITRE ATT&CK. Active scanning: Wordlist scanning, technique t1595.003, 2025. URL https://attack.mitre.org/techniques/T1595/003/. Accessed 2026-04-17.

[4] National Institute of Standards and Technology. Secure hash standard (shs). Technical Report FIPS 180-4, National Institute of Standards and Technology, 2015. URL https://csrc.nist.gov/pubs/fips/180-4/upd1/final.

[5] OWASP Foundation. Forced browsing, 2026. URL https://owasp.org/www-community/attacks/Forced_browsing. Accessed 2026-04-17.

[6] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. doi: 10.1145/317087.317089. URL https://www.schneier.com/academic/archives/1999/05/secure_audit_logs_to.html.